

TROPICAL MEDICAL

Rapport de Projet

Plateforme Médicale Numérique Sénégalaise

Auteur Yakhoub Baby

Email yakhoub.baby@univ-thies.sn

Établissement Université de Thiès

Année 2025 – 2026

Patient

Médecin

Réceptionniste

Pharmacien

Admin

Vue 3 • Node.js • Prisma 5 • SQLite • Socket.io • JWT

Table des matières

Résumé	3
1 Introduction	4
1.1 Contexte	4
1.2 Objectifs	4
1.3 Périmètre du projet	4
2 Architecture du système	5
2.1 Architecture générale	5
2.2 Architecture du frontend	5
2.3 Architecture du backend	6
3 Technologies utilisées	8
3.1 Frontend	8
3.1.1 Vue.js 3	8
3.1.2 Vite 8	8
3.1.3 Pinia	8
3.1.4 Vue Router 4	9
3.1.5 Vue I18n	9
3.1.6 Lucide Vue Next	9
3.1.7 Socket.io Client	9
3.1.8 @vueuse/motion	9
3.1.9 Tailwind CSS 4	9
3.1.10 Montserrat — Google Fonts	9
3.2 Backend	10
3.2.1 Node.js	10
3.2.2 Express.js 5	10
3.2.3 Prisma 5	10
3.2.4 SQLite	11
3.2.5 JSON Web Token (JWT)	11
3.2.6 Socket.io	11

3.2.7	bcryptjs	11
3.2.8	node-cron	11
3.2.9	dotenv	11
4	Modèle de données	12
4.1	Vue d'ensemble	12
4.2	Tables principales	12
5	Fonctionnalités développées	14
5.1	Authentification et contrôle d'accès	14
5.2	Tableau de bord par rôle	14
5.3	Gestion des rendez-vous	14
5.4	File d'attente	15
5.5	Dossiers médicaux	15
5.6	Ordonnances et pharmacie	15
5.7	Hospitalisation et gestion des lits	15
5.8	Facturation et paiements	16
5.9	Téléconsultation	16
5.10	Messagerie interne	16
5.11	Actualités santé	16
5.12	Notifications en temps réel	17
6	Sécurité	18
6.1	Mesures implémentées	18
6.2	Gestion des secrets	18
7	Interface utilisateur et design	19
7.1	Design system	19
7.2	Typographie	19
7.3	Responsive design	19
7.4	Animations	19
7.5	Mode sombre	20
8	Déploiement et gestion de version	21
8.1	Structure du dépôt	21
8.2	Fichier .gitignore	21
8.3	Commandes de lancement	21
9	Conclusion	23
9.1	Bilan	23
9.2	Perspectives d'évolution	23

Résumé

Tropical Medical est une plateforme médicale numérique conçue pour répondre aux besoins des établissements de santé au Sénégal. Elle centralise la gestion des patients, des rendez-vous, des consultations, des dossiers médicaux, de la pharmacie, de l'hospitalisation, de la facturation et de la téléconsultation au sein d'une interface web moderne, accessible sur ordinateur et téléphone mobile.

L'application est construite autour d'une architecture découplée *frontend / backend* : le frontend est développé en **Vue 3** avec l'API Composition, tandis que le backend repose sur **Node.js** avec le framework **Express.js 5** et l'ORM **Prisma 5** pour la gestion de la base de données **SQLite**. La communication en temps réel est assurée par **Socket.io**.

Cinq profils d'utilisateurs sont pris en charge : *Patient, Médecin, Réceptionniste, Pharmacien* et *Administrateur*, chacun disposant d'un espace personnalisé et de fonctionnalités adaptées à son rôle.

Indicateur	Valeur
Nombre de rôles utilisateurs	5
Nombre de modules fonctionnels	14
Nombre de tables en base	22
Communication temps réel	Socket.io
Authentification	JWT (JSON Web Token)

1. Introduction

1.1 Contexte

Le système de santé sénégalais fait face à plusieurs défis opérationnels : gestion manuelle des dossiers patients, manque de coordination entre les différents acteurs médicaux (médecins, pharmaciens, réceptionnistes), et difficulté d'accès aux soins pour les populations éloignées. La digitalisation des processus médicaux représente un levier important pour améliorer la qualité des soins et l'efficacité administrative.

1.2 Objectifs

L'objectif principal de ce projet est de concevoir et développer une application web complète permettant à un établissement de santé de gérer l'ensemble de ses activités cliniques et administratives. Plus spécifiquement, il s'agit de :

- Centraliser les dossiers médicaux des patients sous format numérique ;
- Fluidifier la prise de rendez-vous et la gestion de la file d'attente ;
- Faciliter les prescriptions médicales et le suivi des ordonnances ;
- Assurer la traçabilité de la facturation et des paiements ;
- Permettre la téléconsultation par vidéo ;
- Gérer le stock de médicaments et les lots de la pharmacie ;
- Offrir une messagerie interne entre les acteurs de santé ;
- Diffuser des actualités de santé publique.

1.3 Périmètre du projet

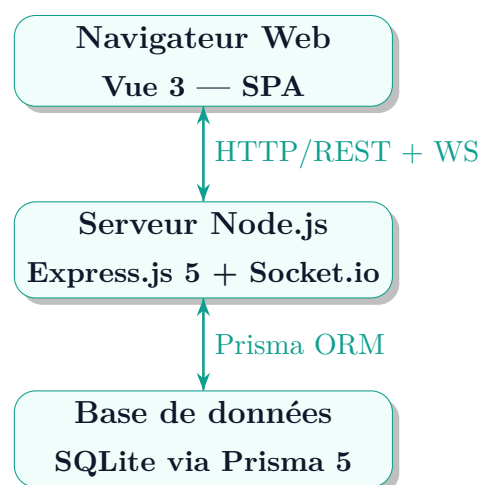
L'application est destinée à être déployée dans des cliniques, hôpitaux de district et centres de santé au Sénégal. Elle prend en charge cinq catégories d'utilisateurs, chacune avec des droits et des interfaces spécifiques.

2. Architecture du système

2.1 Architecture générale

L'application suit une architecture **client-serveur découplée** en trois couches :

1. **Couche présentation** (Frontend) : application web monopage (*Single Page Application*) développée en Vue 3.
2. **Couche métier** (Backend) : API REST sécurisée développée en Node.js / Express.js.
3. **Couche données** (Base de données) : base relationnelle SQLite gérée via l'ORM Prisma 5.



2.2 Architecture du frontend

Le frontend est organisé selon une structure modulaire par rôle :

Listing 2.1 – Structure du répertoire frontend/src/

```
1 src/
2     assets/           # Ressources statiques
3     components/       # Composants réutilisables
4     layout/           # AppSidebar, AppHeader,
                        NotifToast
```

```

5      MessageriePanel.vue
6      composables/      # Hooks réutilisables (useApi,
useDate)
7      layouts/          # Layouts par rôle (Patient,
Medecin, ...)
8      router/           # Configuration Vue Router
9      stores/           # Stores Pinia (auth,
notifications, messagerie)
10     views/            # Pages par rôle
11         medecin/
12         patient/
13         receptionniste/
14         pharmacien/
15         admin/
16     style.css          # Design system global
17     main.js

```

2.3 Architecture du backend

Listing 2.2 – Structure du répertoire backend/src/

```

1  src/
2      app.js            # Point d'entrée Express +
Socket.io
3      config/
4          prisma.js     # Instance Prisma Client
5          constants.js  # Constantes métier (rôles
, statuts)
6      controllers/      # Logique métier par domaine
7          auth.controller.js
8          patient.controller.js
9          medecin.controller.js
10         receptionniste.controller.js
11         pharmacien.controller.js
12         admin.controller.js
13         notifications.controller.js
14         messagerie.controller.js
15         actualite.controller.js
16     middlewares/
17         auth.js        # Vérification JWT

```

```
18         roleCheck.js           # Contrôle d'accès par rô
    le
19         routes/                 # Définition des routes API
20         helpers/
21         notificationHelper.js
22         cron.js                 # Tâches planifiées (node-
    cron)
```


3. Technologies utilisées

3.1 Frontend

3.1.1 Vue.js 3

Vue.js 3 — Framework JavaScript

Vue.js est un framework JavaScript progressif open-source pour la construction d'interfaces utilisateur. La version 3, utilisée dans ce projet, introduit l'**API Composition** (`setup`), qui permet une meilleure organisation du code et une réutilisabilité accrue des logiques métier via des *composables*.

Fonctionnalités Vue 3 utilisées dans le projet :

- `ref()` et `computed()` pour la réactivité ;
- `onMounted()` / `onUnmounted()` pour le cycle de vie ;
- `watch()` pour réagir aux changements d'état ;
- `<Teleport>` pour les modales hors du DOM parent ;
- `<Transition>` et `<TransitionGroup>` pour les animations ;
- `<script setup>` — syntaxe sucre syntaxique de l'API Composition.

3.1.2 Vite 8

Vite est un outil de build ultra-rapide pour le développement web moderne. Il exploite les modules ES natifs du navigateur en mode développement pour éliminer le temps de bundling, ce qui se traduit par des démarrages de serveur quasi-instantanés. En production, il utilise **Rollup** pour générer des bundles optimisés.

3.1.3 Pinia

Pinia est le gestionnaire d'état officiel de Vue 3. Il remplace Vuex avec une API plus simple et une meilleure intégration TypeScript. Dans ce projet, trois stores Pinia sont utilisés :

Store	Fichier	Rôle
useAuthStore	auth.js	Utilisateur connecté, token JWT
useNotificationsStore	notifications.js	Notifications temps réel
useMessagerieStore	messagerie.js	Badge messages non lus

3.1.4 Vue Router 4

Vue Router assure la navigation côté client (SPA). Le routeur est configuré avec des **routes imbriquées par rôle** et des **gardes de navigation** (*beforeEach*) qui vérifient l'authentification JWT et le rôle avant chaque changement de page.

3.1.5 Vue I18n

Vue I18n permet l'internationalisation de l'interface. L'application supporte le **français** et l'**anglais** avec basculement dynamique sans rechargement de page.

3.1.6 Lucide Vue Next

Bibliothèque d'icônes SVG légère (plus de 1 000 icônes). Utilisée pour toutes les icônes de l'interface : menus, boutons, badges, statuts.

3.1.7 Socket.io Client

Utilisé côté frontend pour recevoir les événements temps réel émis par le serveur (nouvelles notifications, nouveaux messages de messagerie).

3.1.8 @vueuse/motion

Bibliothèque d'animations déclaratives pour Vue 3, basée sur les animations CSS et Web Animations API. Utilisée pour les animations d'entrée des éléments.

3.1.9 Tailwind CSS 4

Framework CSS utilitaire intégré via le plugin Vite. Complète le design system personnalisé de l'application pour certaines mises en page.

3.1.10 Montserrat — Google Fonts

Toutes les polices de l'application utilisent la famille **Montserrat** (poids 300 à 900), importée depuis Google Fonts. Cette police sans-serif géométrique confère à l'interface un aspect moderne et professionnel.

3.2 Backend

3.2.1 Node.js

Node.js est un environnement d'exécution JavaScript côté serveur, basé sur le moteur V8 de Chrome. Son modèle non-bloquant et orienté événements le rend particulièrement adapté aux applications web nécessitant de nombreuses connexions simultanées.

3.2.2 Express.js 5

Express.js est le framework web minimaliste le plus utilisé avec Node.js. La version 5, adoptée dans ce projet, apporte notamment la gestion native des erreurs asynchrones sans *try/catch* explicite.

L'API REST expose les routes suivantes :

Préfixe	Domaine
/api/auth	Authentification
/api/patient	Espace patient
/api/medecin	Espace médecin
/api/receptionniste	Espace réceptionniste
/api/pharmacien	Espace pharmacien
/api/admin	Administration
/api/notifications	Notifications
/api/messagerie	Messagerie interne

3.2.3 Prisma 5

Prisma est un ORM (*Object-Relational Mapper*) de nouvelle génération pour Node.js. Il génère automatiquement un client TypeScript/JavaScript typé à partir d'un schéma déclaratif (`schema.prisma`), ce qui élimine les requêtes SQL manuelles et réduit les erreurs.

Fonctionnalités Prisma utilisées :

- Migrations automatiques (`prisma migrate dev`);
- Peuplement de la base (`prisma db seed`);
- Studio visuel (`prisma studio`) pour inspecter les données;
- Relations `@relation`, clés étrangères, contraintes `@unique`.

3.2.4 SQLite

SQLite est utilisé comme base de données relationnelle. Légère et sans serveur dédié, elle est idéale pour le développement et les petites structures. La base de données est un seul fichier (`dev.db`) stocké localement.

3.2.5 JSON Web Token (JWT)

L'authentification repose sur les **JWT** (*JSON Web Tokens*). Lors de la connexion, le serveur génère un token signé contenant l'identifiant et le rôle de l'utilisateur. Ce token est envoyé dans l'en-tête **Authorization: Bearer <token>** à chaque requête et vérifié par le middleware `auth.js`.

3.2.6 Socket.io

Socket.io implémente la communication bidirectionnelle en temps réel entre le serveur et les clients via WebSocket (avec fallback HTTP long-polling). Utilisations dans le projet :

- Réception des **notifications** instantanées pour chaque utilisateur connecté (chambre `user:<id>`);
- Émission d'un événement `nouveau_message` lors de l'envoi d'un message de messagerie.

3.2.7 bcryptjs

Bibliothèque de hachage des mots de passe avec l'algorithme **bcrypt**. Tous les mots de passe sont stockés sous forme hachée (jamais en clair) avec un facteur de coût de 10 itérations.

3.2.8 node-cron

Planificateur de tâches CRON pour Node.js. Utilisé pour exécuter des tâches automatiques à intervalles réguliers (rappels de rendez-vous, nettoyage de données expirées, etc.).

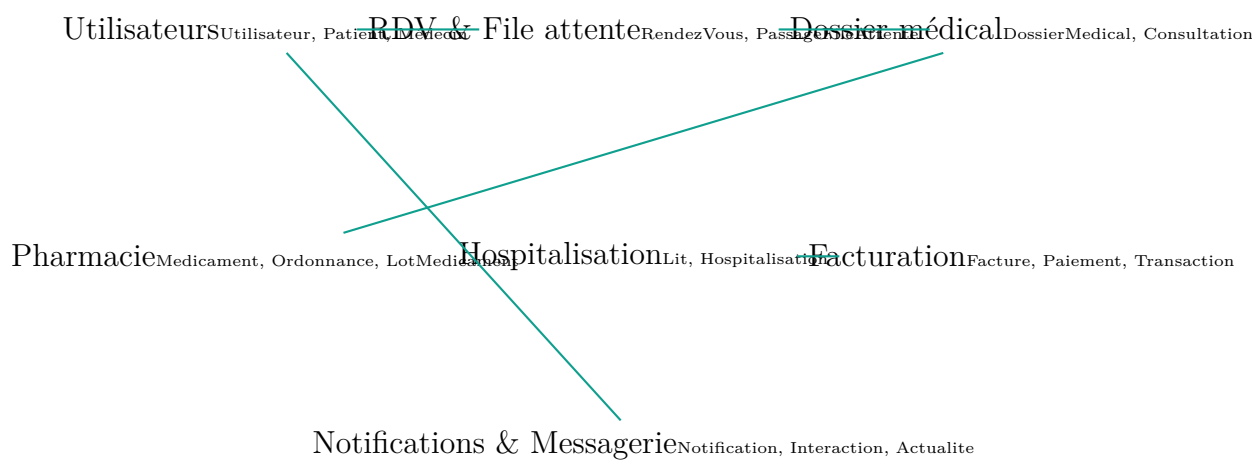
3.2.9 dotenv

Charge les variables d'environnement depuis le fichier `.env` dans `process.env`. Permet de séparer la configuration sensible (secrets JWT, URL de base de données) du code source.

4. Modèle de données

4.1 Vue d'ensemble

La base de données comporte **22 tables** réparties en six domaines fonctionnels. Le schéma est défini dans `backend/prisma/schema.prisma`.



4.2 Tables principales

Table	Domaine	Description
Utilisateur	Auth	Compte de base (nom, email, rôle, mot de passe haché)
Patient	Profil	Données médicales du patient (groupe sanguin, allergies, antécédents)
Medecin	Profil	Spécialité, numéro d'ordre, tarif consultation
RendezVous	Planning	Date, type (présentiel/téléconsultation), statut, motif
PassageFileAttenteFlux		Gestion de la file avec niveau d'urgence

Table	Domaine	Description
DossierMedical	Clinique	Dossier central, relié aux consultations et ordonnances
Consultation	Clinique	Anamnèse, examen clinique, diagnostic, traitement
Ordonnance	Pharmacie	Prescriptions avec lignes de médicaments
Medicament	Pharmacie	Catalogue médicaments (stock, prix, DCI)
LotMedicament	Pharmacie	Traçabilité des lots (numéro, expiration, quantité)
Lit	Hospit.	Numéro, chambre, étage, type, statut
Hospitalisation	Hospit.	Admission, sortie, coût par jour, motif
Facture	Finances	Montant total, part assurance, part patient, statut
Paielement	Finances	Mode de paiement, référence, montant
Teleconsultation	Numérique	URL de la salle, plateforme, durée
Notification	Alertes	Titre, message, canal, lu/non lu
Interaction	Messagerie	Message, expéditeur, destinataire, lu/non lu
Actualite	Info santé	Titre, contenu, catégorie, auteur, publiée

5. Fonctionnalités développées

5.1 Authentification et contrôle d'accès

Le système d'authentification repose sur :

- **Inscription/Connexion** avec email et mot de passe haché par bcrypt ;
- Génération d'un **token JWT** valide 24 heures à la connexion ;
- Stockage du token dans `localStorage` côté client ;
- Middleware `auth.js` vérifiant la signature du token à chaque appel API ;
- Middleware `roleCheck.js` autorisant ou refusant l'accès selon le rôle de l'utilisateur (PATIENT, MEDECIN, RECEPTIONNISTE, PHARMACIEN, ADMIN).

5.2 Tableau de bord par rôle

Chaque rôle dispose d'un tableau de bord personnalisé avec des KPI (*Key Performance Indicators*) animés et des raccourcis vers les modules principaux :

Rôle	KPI affichés
Patient	Prochains RDV, factures en attente, ordonnances actives
Médecin	File d'attente, consultations du jour, urgences
Réceptionniste	RDV du jour, lits libres/occupés, paiements récents
Pharmacien	Stock faible, ordonnances à délivrer, lots expirants
Admin	Nombre d'utilisateurs par rôle, activité globale

5.3 Gestion des rendez-vous

- Prise de RDV en ligne par le patient (choix médecin, date, motif) ;
- Types de RDV : présentiel ou téléconsultation ;
- Statuts : EN_ATTENTE, CONFIRME, ANNULE, TERMINE ;
- Filtres par statut avec onglets colorés.

5.4 File d'attente

Gestion en temps quasi-réel des patients en attente de consultation :

- Enregistrement à l'arrivée avec niveau d'urgence (FAIBLE, MODERE, URGENT, CRITIQUE) ;
- Appel des patients par le médecin ;
- Conversion automatique en consultation à l'appel.

5.5 Dossiers médicaux

Chaque patient possède un dossier médical unique contenant :

- Historique des consultations (anamnèse, diagnostic, traitement) ;
- Vaccinations avec dates de rappel ;
- Examens médicaux (biologie, radiologie, imagerie) ;
- Antécédents médicaux et chirurgicaux ;
- Maladies chroniques et allergies.

5.6 Ordonnances et pharmacie

- Création d'ordonnances par le médecin avec sélection de médicaments depuis le catalogue ;
- Impression PDF de l'ordonnance dans une fenêtre dédiée ;
- Délivrance par le pharmacien avec mise à jour du stock ;
- Gestion des lots de médicaments (numéro, date d'expiration, quantité) ;
- Alertes automatiques pour les stocks faibles.

5.7 Hospitalisation et gestion des lits

- Grille visuelle des lits par étage avec code couleur (Libre / Occupé / Réserve / Maintenance) ;
- Admission d'un patient sur un lit libre directement depuis la grille ;
- Suivi du coût par jour et calcul automatique de la facture de séjour ;
- Libération du lit à la sortie.

5.8 Facturation et paiements

- Génération automatique d'une facture après consultation ou hospitalisation ;
- Calcul de la part assurance et de la part patient ;
- Modes de paiement : espèces, carte bancaire, Mobile Money (Orange Money, Wave, Free Money) ;
- Suivi des statuts : `EN_ATTENTE`, `PARTIELLEMENT_PAYE`, `PAYE`.

5.9 Téléconsultation

- Planification d'une séance de vidéo-consultation lors de la prise de RDV ;
- Génération d'un lien de salle **Jitsi Meet** ;
- Accès à la salle depuis le tableau de bord du médecin et du patient ;
- Enregistrement de la durée de la session.

5.10 Messagerie interne

Système de messagerie directe entre les acteurs de santé :

- Liste des conversations avec dernière message et compteur non lus ;
- Envoi et réception de messages en temps (quasi) réel via **polling toutes les 8 secondes** ;
- Badge sur le menu latéral indiquant le nombre de messages non lus, mis à jour toutes les 15 secondes via un store Pinia ;
- Remise à zéro du badge à l'ouverture de la messagerie ;
- Accessible aux trois rôles : Patient, Médecin, Réceptionniste.

5.11 Actualités santé

- Publication d'actualités de santé publique par la réceptionniste (campagnes de vaccination, journées mondiales, dépistages...) ;
- Catégories : `INFO`, `DON_SANG`, `VACCINATION`, `SENSIBILISATION`, `EVENEMENT`, `URGENCE` ;
- Consultation par le patient avec filtres par catégorie et modal de lecture complète ;
- Gestion CRUD (créer, lire, modifier, supprimer, publier/dépublier).

5.12 Notifications en temps réel

- Notification in-app à chaque événement important (RDV confirmé, nouveau message, résultat d'examen disponible) ;
- Connexion Socket.io au chargement du dashboard ;
- Affichage sous forme de toast (*pop-up* éphémère) et dans un panneau de notifications persistant.

6. Sécurité

6.1 Mesures implémentées

Mesure	Description
Hachage des mots de passe	bcryptjs avec 10 rounds de salage
Authentification stateless	JWT signé avec secret configurable
Contrôle d'accès	Middleware roleCheck par route API
CORS configuré	Origines autorisées limitées au frontend
Variables sensibles	Secrets dans <code>.env</code> (exclu du dépôt Git)
Validation des entrées	Vérification côté serveur avant chaque écriture
Séparation des espaces	Chaque rôle n'accède qu'à ses propres données

6.2 Gestion des secrets

Les données sensibles sont stockées dans le fichier `.env` qui n'est **jamais versionné** (présent dans `.gitignore`). Un fichier `.env.example` documente les variables nécessaires sans les valeurs réelles :

Listing 6.1 – Contenu de backend/.env.example

```
1 DATABASE_URL="file:./dev.db"
2 JWT_SECRET=remplacer_par_une_cle_secrete_longue_et_aleatoire
3 JWT_EXPIRES_IN=24h
4 PORT=5000
5 NODE_ENV=development
6 FRONTEND_URL=http://localhost:5173
```

7. Interface utilisateur et design

7.1 Design system

Un design system cohérent est défini dans `frontend/src/style.css` via des **variables CSS** (*custom properties*) :

Listing 7.1 – Extrait des variables CSS globales

```
1 :root {  
2   --color-primary:    #0E9F8E;    /* Teal medical */  
3   --color-danger:     #EF4444;     /* Rouge urgence */  
4   --color-warning:    #F59E0B;     /* Ambre alerte */  
5   --font-sans:        'Montserrat', system-ui, sans-serif;  
6   --font-display:     'Montserrat', sans-serif;  
7   --radius-card:     1.125rem;    /* Arrondi des cartes */  
8 }
```

7.2 Typographie

Toute l'application utilise la police **Montserrat** (Google Fonts), disponible en poids 300 (Light) à 900 (Black). Ce choix offre une lisibilité optimale sur écran et une apparence moderne et professionnelle adaptée au secteur médical.

7.3 Responsive design

L'interface s'adapte aux différentes tailles d'écran grâce aux *media queries* CSS. La barre latérale se replie automatiquement sur mobile (en dessous de 1024 px) et un bouton hamburger permet de l'afficher en superposition.

7.4 Animations

L'application intègre plusieurs niveaux d'animations :

- **Entrées en cascade** (*stagger*) : les cartes apparaissent séquentiellement avec un délai croissant ;
- **Ressort** (*spring*) : courbe `cubic-bezier(.34,1.56,.64,1)` pour un effet rebond naturel ;
- **Bande lumineuse** (*shine*) : reflet animé sur les barres colorées des cartes statistiques ;
- **Pulse** : clignotement doux sur les badges « Occupés » pour attirer l'attention ;
- **Compteurs animés** : les chiffres des KPI s'incrémentent au chargement (`AnimCounter`).

7.5 Mode sombre

L'application supporte un mode sombre activable depuis les paramètres utilisateur. Les couleurs basculent via les variables CSS en appliquant la classe `dark` sur l'élément `<html>`.

8. Déploiement et gestion de version

8.1 Structure du dépôt

Listing 8.1 – Structure racine du projet

```
1 tropical-medical/  
2     backend/           # API Node.js  
3         src/  
4         prisma/  
5         .env.example  
6         package.json  
7     frontend/         # SPA Vue 3  
8         src/  
9         index.html  
10        package.json  
11    .gitignore          # Exclut .env, node_modules, dev.db  
12    README.md
```

8.2 Fichier .gitignore

Le fichier `.gitignore` à la racine exclut du dépôt Git :

- Les dossiers `node_modules/` (dépendances);
- Les fichiers `.env` (secrets);
- La base de données SQLite (`*.db`, `*.db-journal`);
- Le client Prisma généré (`backend/generated/`);
- Le build de production (`frontend/dist/`).

8.3 Commandes de lancement

Listing 8.2 – Démarrage du projet en développement

```
1  # Backend
2  cd backend
3  npm install
4  npx prisma migrate dev
5  node prisma/seed.js      # Donnees initiales
6  npm run dev              # Port 5000
7
8  # Frontend (autre terminal)
9  cd frontend
10 npm install
11 npm run dev              # Port 5173
```

9. Conclusion

9.1 Bilan

Ce projet a abouti à la réalisation d'une plateforme médicale complète et fonctionnelle, couvrant l'ensemble du cycle de vie d'un patient au sein d'un établissement de santé. L'application **Tropical Medical** propose :

- Une architecture moderne et maintenable (séparation frontend/backend) ;
- Une interface intuitive adaptée à cinq profils d'utilisateurs distincts ;
- Des fonctionnalités temps réel via Socket.io ;
- Un système de sécurité robuste basé sur JWT et le contrôle d'accès par rôle ;
- Un design system cohérent avec animations et thème sombre.

9.2 Perspectives d'évolution

Plusieurs améliorations peuvent être envisagées pour les versions futures :

1. Migration vers **PostgreSQL** ou **MySQL** pour la production ;
2. Intégration d'une **API de paiement mobile** réelle (Orange Money, Wave) ;
3. Module de **rapports et statistiques** avec graphiques avancés ;
4. Application mobile native (**React Native** ou **Capacitor**) ;
5. Déploiement sur un serveur cloud (**VPS** ou **PaaS**).

Bibliographie

- [1] Evan You et al., *Vue.js 3 — The Progressive JavaScript Framework*, <https://vuejs.org>, 2024.
- [2] Eduardo San Martin Morote, *Pinia — The intuitive store for Vue.js*, <https://pinia.vuejs.org>, 2024.
- [3] Prisma Inc., *Prisma — Next-generation Node.js and TypeScript ORM*, <https://www.prisma.io>, 2024.
- [4] OpenJS Foundation, *Express.js — Fast, unopinionated, minimalist web framework for Node.js*, <https://expressjs.com>, 2024.
- [5] Socket.io Contributors, *Socket.IO — Bidirectional and low-latency communication*, <https://socket.io>, 2024.
- [6] M. Jones, J. Bradley, N. Sakimura, *RFC 7519 — JSON Web Token (JWT)*, IETF, 2015.
- [7] Evan You et al., *Vite — Next Generation Frontend Tooling*, <https://vite.dev>, 2024.
- [8] D. Richard Hipp, *SQLite — A self-contained, serverless SQL database engine*, <https://sqlite.org>, 2024.